

Abgeschlossen und anschlussfähig

Eine bedingte Formalisierung evidenzgebundener
Zustandsarchitekturen mit einem Lean/Lake-Prüfpaket

Ingolf Lohmann: Architekturidee, Ausgangstext und Fragestellung
OpenAI Codex: KI-assistierte Ausarbeitung und neue Formalisierung
15. September 2026 · Version 0.1 · fachlich noch nicht begutachtet

Zusammenfassung

Kann ein System in sich abgeschlossen und zugleich für neue Erkenntnisse offen sein? Die Antwort hängt davon ab, was abgeschlossen bedeutet. Dieser Artikel trennt den mathematischen Abschluss eines Modellbestands von operativer Fortsetzung, aktueller Evidenz und physischer Wirkung. Für ein explizites Übergangsmodell wird ein Abschlussoperator C über endlichen zulässigen Ableitungen definiert. Seine Monotonie und Idempotenz sowie eine bedingte echte Erweiterung werden elementar hergeleitet. Ein bereits abgeschlossener Bestand ist damit nicht notwendig maximal.

Die Übertragung auf QIK-VRT erfolgt als präzise Architekturinterpretation: Aussagen bleiben an Subjekt und Version gebunden; zulässige Verkettung verlangt passende Schnittstellen. Fortsetzung benötigt eine eigene Nachfolgerannahme. Ein Gegenmodell zeigt, warum bloße Viabilität diese nicht ersetzt. Für Hardware und physische Prozesse bleiben Implementierungs- und Beobachtungskorrespondenz gesonderte Nachweispflichten.

Ausführungsstatus: Der neue Lean-Code und das Prüfpaket sind vorhanden. Der lokale Kernellauf ist am Start der Lean-Umgebung gescheitert, noch vor der Prüfung der neuen Quellen. **FORMAL_PENDING_KERNEL** ist deshalb der Status dieses Kandidaten. Ein erfolgreicher Lean-Beweis, ein fertiger Chip oder eine Publikationsfreigabe werden nicht behauptet.

Schlagwörter: QIK-VRT; Abschlussoperator; Zustandsübergang; Anschlussfähigkeit; Evidenzbindung; Lean 4; formale Verifikation; Refinement.

1 Fragestellung und Beitrag

1.1 Die produktive Unterscheidung

Der von Ingolf Lohmann formulierte Ausgangsgedanke verbindet Beobachten, Binden, Handeln und Rückbeobachten zu einem fortsetzbaren Erkenntnisprozess. In der vorliegenden Rekonstruktion bedeutet die Formel $\text{Observe} \rightarrow \text{Bind} \rightarrow \text{Act} \rightarrow \text{Readback}$ keine automatische Wahrheitsgarantie. Sie benennt Verpflichtungen: den Ausgangszustand bestimmen, den Bezug einer Handlung festlegen, ihre Zulässigkeit prüfen und anschließend das tatsächlich beobachtete Ergebnis einordnen.

Eine abgeschlossene Begründung kann die Grundlage einer neuen Begründung bilden. Das ist kein Widerspruch, solange Abschluss relativ zu einem bestimmten Bestand, einer Regelmenge und einer Geltungsgrenze verstanden wird. Die Aussage, ein System habe sämtliche relevanten Eigenschaften seiner gesamten Umwelt bewiesen, wäre wesentlich stärker und folgt daraus nicht.

1.2 Drei verschiedene Bedeutungen von Abschluss

Begriff	Präzise Bedeutung	Was daraus nicht folgt
Modellabschluss	Unter festgelegten Regeln entstehen bei erneuter Abschlussbildung keine zusätzlichen Elemente.	Dass alle möglichen Grundlagen bereits enthalten sind.
Operativer Abschluss	Ein abgegrenzter Arbeitsauftrag hat seine definierten Nachweise erbracht.	Dass jede Folgehandlung zulässig oder ausführbar ist.
Empirischer Nachweis	Beobachtungen tragen eine konkret begrenzte Aussage unter offengelegten Messbedingungen.	Unfehlbarkeit des Sensors oder universelle Kausalität.

1.3 Einordnung in vorhandene Arbeit

Das Prüfpaket verwendet unverändert die Repository-Module `OperationalContinuation` und `ConnectabilitySimulation`. Dort sind endliche viable Fortsetzung sowie viable Spuren und `Simulation` bereits definiert. Der neue Zusatz ergänzt die Verkettung solcher Spuren, einen darauf aufgebauten Abschlussoperator und explizite Gegenmodelle. Es wird kein konkurrierendes Übergangsmodell eingeführt. [R1, R2]

Abschlussoperatoren und die Komposition endlicher Übergänge sind etablierte mathematische Konstruktionen. Der Beitrag dieses Artikels ist ihre nachvollziehbare Zuordnung zur vorliegenden QIK-VRT-These, nicht ein Beweis historischer Priorität oder ein Anspruch auf die Erfindung von Rückkopplung. Eine vollständige Neuheitsprüfung gehört nicht zum ausgeführten Umfang.

Auch in verteilten Systemen muss eine reine Zeitordnung von einer durch Prozess- und Nachrichtenbeziehungen definierten Ordnung unterschieden werden. Lamports happens-before-Beziehung liefert hierfür einen klassischen Bezugspunkt; sie beweist jedoch keine beliebige physische Ursache-Wirkungs-Behauptung. [R5]

2 Modell und Begriffe

Seien X und L Typen von Zuständen beziehungsweise Bezeichnungen. Ein System T besteht aus einer Relation $\text{step} : X \times L \times X \rightarrow \text{Prop}$ und einem Prädikat $\text{viable} : X \rightarrow \text{Prop}$. Ein gültiger Einzelschritt verlangt Viabilität an beiden Endpunkten und eine gültige Übergangsrelation.

$$\text{ValidStep}(x, l, y) \Leftrightarrow \text{viable}(x) \wedge \text{step}(x, l, y) \wedge \text{viable}(y).$$

Das vorhandene Modell $\text{ViableTrace}(T, x, w, y)$ beschreibt eine endliche Spur mit der Bezeichnungsfolge w . Die leere Spur verlangt $x = y$ und $\text{viable}(x)$. Eine nichtleere Spur besteht aus einem gültigen Schritt und einer gültigen Restspur. Es handelt sich um ein Modellprädikat, nicht um eine Behauptung, die beschriebene Ausführung sei tatsächlich geschehen.

2.1 Abschluss eines Anfangsbestands

Für einen Anfangsbestand S , dargestellt als Prädikat auf X , definieren wir:

$$C_T(S)(x) \Leftrightarrow \exists a, w: S(a) \wedge \text{ViableTrace}(T, a, w, x).$$

C enthält genau die von S aus durch endliche viable Spuren ableitbaren Elemente. Wird T als Regelgraph interpretiert, ist dies ein Abschluss unter diesen Regeln. Wird T als operatives Modell interpretiert, beschreibt C mögliche Modellzustände. Beides darf nicht mit einer Sammlung bereits gemessener Wirkungen gleichgesetzt werden.

Die Domäne für einen extensiven Abschlussoperator sind hier die Bestände S mit $S \subseteq V$, wobei V die Menge viabler Zustände ist. Diese Einschränkung ist notwendig: Ein nichtviabler Anfangszustand besitzt im verwendeten Modell nicht einmal eine gültige leere Spur.

2.2 Fixpunkt und Erweiterung

$$\text{Closed}_T(E) \Leftrightarrow \forall x: C_T(E)(x) \Leftrightarrow E(x).$$

Das Wort Fixpunkt bezeichnet ausschließlich diese Gleichheit von Prädikaten im extensionalen Sinn. Es bezeichnet weder ein selbstbestätigendes Programm noch einen abgeschlossenen Nachweis über die Welt. Die Lean-Fassung verwendet punktweise Äquivalenz, sodass keine Gleichheit von Funktionen zusätzlich benötigt wird.

$$\text{Extend}(S, z)(x) \Leftrightarrow S(x) \vee x = z.$$

Eine echte Erweiterung benötigt ein viables z außerhalb des bisherigen Abschlusses. Ob ein solches z existiert und als neue empirische Grundlage angenommen werden darf, ist eine eigene Frage. Der Erweiterungssatz setzt es voraus; er erzeugt es nicht aus dem Begriff Abschluss.

3 Abschlusssatz und Beweise

Satz 1: Verkettung

Aus $\text{ViableTrace}(T, a, u, b)$ und $\text{ViableTrace}(T, b, v, c)$ folgt $\text{ViableTrace}(T, a, u ++ v, c)$.

Beweis: Induktion über u . Für die leere Spur ist $a = b$, und die zweite Spur genügt. Im nichtleeren Fall bleibt der erste gültige Schritt erhalten; die Induktionsannahme verkettet die erste Restspur mit v . Entscheidend ist der identische Anschlusspunkt b , nicht bloß ein späterer Zeitstempel.

Satz 2: Abschlussoperator

Für viable Anfangsbestände gilt $S \subseteq C(S)$. Der Zeuge ist die leere Spur am jeweiligen Anfangselement. Aus $S \subseteq S'$ folgt $C(S) \subseteq C(S')$: Dieselben Anfangs- und Spurzeugen bleiben gültig. Außerdem liegen alle Elemente von $C(S)$ in V , weil jede viable Spur an einem viablen Zustand endet.

$$C(C(S)) = C(S).$$

Beweis der Idempotenz: Ein Element von $C(C(S))$ wird von einem Zwischenzustand b erreicht, der selbst von S erreichbar ist. Satz 1 verkettet beide endlichen Spuren zu einer Spur von S . Umgekehrt liegt jedes x aus $C(S)$ über die leere Spur wieder in $C(C(S))$, da x viable ist. Damit ist $C(S)$ ein Fixpunkt von C .

Satz 3: Abgeschlossen und dennoch echt erweiterbar

Seien $z \in V$ und $z \notin C(S)$. Dann sind $C(S)$ und $C(S \cup \{z\})$ beide abgeschlossen, und es gilt:

$C(S)$ ist eine echte Teilmenge von $C(S \cup \{z\})$.

Beweis: Beide Bestände sind nach Satz 2 Fixpunkte. Monotonie erhält jedes bisherige Element. Die leere Spur zeigt $z \in C(S \cup \{z\})$; die Voraussetzung $z \notin C(S)$ zeigt, dass die Inklusion echt ist. Damit ist der behauptete Zusammenhang für dieses Modell hergeleitet. Die Voraussetzungen bleiben Teil der Aussage.

Ein konkreter Zeuge ist $X = \{\text{false}, \text{true}\}$, viable stets wahr und step stets falsch. Dann ist $C(S) = S$. Der Bestand $\{\text{false}\}$ ist abgeschlossen und lässt sich zu $\{\text{false}, \text{true}\}$ echt erweitern. Der Zeuge widerlegt die Gleichsetzung von abgeschlossen mit maximal; er belegt nicht, dass jede Erweiterung operativ sinnvoll ist.

Lean-Zuordnung: `trace_append`; `closure_extensive`; `closure_monotone`; `closure_viable`; `closure_idempotent`; `closure_is_closed`; `closed_but_extendible`; `no_rules_closure`; `strict_extension_witness`. Der aktuelle Status dieser neuen Quellen ist `FORMAL_PENDING_KERNEL`.

4 Fortsetzung ist eine eigene Pflicht

Der Abschlussoperator beantwortet eine Mengenfrage. Ob ein technischer Prozess weiterlaufen kann, ist eine andere Frage. Das wiederverwendete ContinuationSystem besitzt eine Verbindungsrelation connects und ein Viabilitätsprädikat. Eine viable Verbindung verlangt $\text{viable}(x)$, $\text{connects}(x,y)$ und $\text{viable}(y)$. [R1]

$\text{Survives}(0, x) \Leftrightarrow \text{viable}(x)$.

$\text{Survives}(n+1, x) \Leftrightarrow \exists y: \text{ViableConnection}(x,y) \wedge \text{Survives}(n,y)$.

Satz 4: Bedingte endliche Fortsetzbarkeit

Angenommen, jeder viable Zustand besitzt mindestens einen viablen Nachfolger. Dann gilt für jedes endliche n und jeden viablen Anfangszustand x : $\text{Survives}(n,x)$. Beweis: Induktion über n . Der Nullfall ist die Viabilitätsvoraussetzung. Im Nachfolgerfall liefert die Annahme einen viablen Anschluss y ; auf y wird die Induktionsannahme angewandt.

$\forall x \in V: \exists y: \text{ViableConnection}(x,y)$

$\Rightarrow \forall n \in \mathbb{N}, x \in V: \text{Survives}(n,x)$.

Dies ist kein Satz über die tatsächliche Laufzeit eines realen Rechners. Er garantiert auch nicht, dass eine konkrete Auswahlstrategie den vorhandenen Nachfolger findet.

Unbeschränkte reale Ausführung würde zusätzliche Ressourcen-, Umgebungs- und gegebenenfalls Fairnessbedingungen benötigen. Der Unterschied zwischen beliebig langen endlichen Möglichkeiten und einem real ausgeführten unendlichen Prozess bleibt bestehen.

Gegenmodell: der sichere Stillstand

Das Modell deadlock hat einen einzigen Zustand $()$, viable ist wahr, connects stets falsch. Damit ist der Zustand viable, aber $\text{Survives}(1,())$ falsch. Die Implikation ‚lokale Gültigkeit bedeutet nächste zulässige Handlung‘ ist folglich ohne Zusatzannahme ungültig. Ein owner-bound HOLD kann praktisch genau diese Grenze ausdrücken: Der bisherige Nachweis bleibt erhalten, während eine nächste Handlung noch nicht zugelassen ist.

Nichtleerer Fortsetzungszeuge

Im Modell counterSystem ist $X = \mathbb{N}$, jeder Zustand viable und y genau dann Nachfolger von x , wenn $y = x+1$. Die Nachfolgerannahme wird hier durch die konkrete Funktion $x \mapsto x+1$ erfüllt. Für jedes endliche n folgt die entsprechende Modellfortsetzung. Dieser Zeuge verhindert eine ausschließlich unerfüllbare Voraussetzung im Beispiel; er zertifiziert keinen physischen Zähler und keinen Chip.

Die wiederverwendeten Simulationssätze zeigen zusätzlich: Werden Viabilität und Schritte unter einer Relation erhalten, lassen sich endliche viable Spuren übertragen. Das verbindet Modelle unter expliziten Bedingungen. Es beweist nicht, dass zwei beliebige Implementierungen dieselbe Bedeutung besitzen. [R2]

5 Identität, Evidenz und Wirkung

5.1 Zustandsbindung ist keine globale Behauptung

Der Zusatzcode modelliert ein Subjekt als Tupel aus Artefakt-, Revisions- und Policy-Kennung. Ein Receipt trägt dieses Subjekt und ein Validierungsprädikat. $\text{Bound}(r,s)$ bedeutet: Die Subjektkennung stimmt exakt überein, und das Validierungsprädikat gilt. Die Validierung selbst bleibt eine Voraussetzung; das Modell prüft keinen Sensor, keinen Hashalgorithmus und keine reale Unterschrift.

$$\text{Bound}(r,s) \wedge s \neq t \Rightarrow \neg \text{Bound}(r,t).$$

Beweis: Wäre dasselbe Receipt zugleich an t gebunden, ergäben beide Gleichheiten $s = t$, im Widerspruch zur Voraussetzung. In der Praxis verhindert dieser Typ von Bedingung, dass ein Head-spezifischer Nachweis allein wegen eines grünen Status auf einen anderen Head übertragen wird. Die Architektur muss die Vergleichsdaten tatsächlich prüfen und darf den Check nicht nur als frei setzbares Feld annehmen.

Nichtübertragbarkeit eines Receipts bedeutet nicht, dass bei jeder Änderung alle Beweise verworfen werden. Ein expliziter Nachweis der Unverändertheit des relevanten Teilumfangs kann Wiederverwendung rechtfertigen. Dann trägt dieser zusätzliche Nachweis die Übertragung. Bloße Versionsähnlichkeit genügt nicht.

5.2 Transport ist schwächer als Wirkung

Die universelle Formel ‚für beliebige Bool-Werte transport und effect folgt aus $\text{transport} = \text{true}$ auch $\text{effect} = \text{true}$ ‘ ist falsch. Der Zeuge $\text{transport} = \text{true}$, $\text{effect} = \text{false}$ genügt. Diese elementare Negativaussage bildet die Architekturgrenze $\text{TRANSPORT_ACK} \neq \text{EFFECT_ACK}$ ab; sie ersetzt noch keine technische Spezifikation eines Übertragungsprotokolls.

5.3 Rückbeobachtung ist notwendig, aber nicht allein hinreichend

Ein Akteur kann einen Befehl empfangen haben, ohne die gewünschte Wirkung zu erzeugen. Umgekehrt kann die gewünschte Wirkung nach dem Befehl auch aus einer anderen Ursache eintreten. Deshalb muss eine belastbare Wirkungsbehauptung neben einer passenden Rückbeobachtung den richtigen Gegenstand, ein zulässiges Zeitfenster, die Messbedingungen und konkurrierende Erklärungen berücksichtigen.

Für ein einfaches Messmodell $y = H(x) + \eta$ kann eine Schranke $|\eta| \leq \varepsilon$ Bestandteil der Annahmen sein. Die zusätzliche Aussage ‚die beobachtete Abweichung liegt innerhalb ε ‘ ist nur so belastbar wie die Referenzbindung und die empirische Rechtfertigung dieser Schranke. Lean könnte die bedingte Folgerung prüfen, aber ε nicht ohne Messdaten für ein konkretes Gerät beglaubigen.

Evidenzgraphen ordnen somit Begründungen. Dass eine Kante technisch gespeichert wurde, beweist noch nicht, dass die physische Kausalbeziehung, die sie bezeichnet, zutrifft.

6 Anschluss an Hardware und SoC

Die gemeinsame Beschreibung von Hardware und Software als Zustandsübergangssystem liefert eine mögliche Abstraktionsschnittstelle. Sie macht die Implementierungen nicht identisch. Ein Prozessorzustand, ein Softwareprozess und ein Messwert besitzen unterschiedliche Beobachtungs- und Fehlergrenzen. Gerade diese Grenzen müssen in einer tragfähigen Komposition explizit bleiben.

Für eine SoC-Realisierung von QIK-VRT ist zunächst eine konkrete, endliche und effektiv auswertbare Instanz des Kontrollkerns festzulegen. Der generische Lean-Typ `Nat` oder ein beliebiges Prädikat in `Prop` bezeichnet weder automatisch begrenzte Register noch eine synthetisierbare Schaltung. Die These ‚dieser konkrete Kern ist realisierbar‘ verlangt einen Implementierungszeugen und eine passende Verfeinerungsrelation.

Nachweisstufe	Erforderliches Ergebnis	Hier erbracht?
Abstrakter Vertrag	Zustände, Bedingungen, Übergänge und Grenzen explizit modellieren.	Ja, für den begrenzten Artikelumfang.
Lean-Kernel	Genau diese neuen Quellen ohne unbelegte Beweisannahmen prüfen.	Nein: Startfehler der Umgebung.
Implementierung	Endliche Datenpfade und eine konkrete RTL-/Programminstanz mit Refinement.	Nein; kein Chip geliefert.
Synthese / Timing	Ressourcen, Takt, Reset, Nebenläufigkeit und Fehlerpfade für ein Ziel prüfen.	Nein.
Realer Effekt	Frischen unabhängigen Readback der konkreten Hardware oder des Clients binden.	Nein.

Bei einer sicherheitsbezogenen Verfeinerung muss insbesondere jeder mögliche relevante Implementierungsschritt durch das abstrakte Modell gedeckt sein. Die Gegenrichtung - ein Implementierungsmodell kann jeden geforderten Spezifikationsschritt nachbilden - betrifft dagegen die Realisierbarkeit des geforderten Verhaltens. Eine einzelne Simulationsrichtung darf nicht stillschweigend für beide Ansprüche stehen.

Ein geeigneter Weg wäre, den begrenzten Kontrollkern zunächst in ausführbarer Form zu definieren, die Übergangstabelle gegen die Spezifikation zu vergleichen, den RTL-Entwurf zu verfeinern und anschließend Synthese sowie Hardware-Readback zu binden. Testanzahlen müssen dabei klar als Zustände, Assertions, Testfälle oder Ausführungsläufe ausgewiesen werden. Viele Tests eines Referenzmodells beweisen nicht sämtliche Eigenschaften eines Transputers.

Ein IP-bootfähiges Image würde zusätzlich Manifest, Format, Zielarchitektur, Integrität, sicheren Empfang, Startverhalten und den tatsächlich beobachteten Clientzustand benötigen. Dieser Artikel erzeugt kein solches Image und keine dafür gültige Download-URL.

7 Reproduzierbarkeit und Ausführung

7.1 Exakter Quellumfang

Das Paket enthält EvidenceClosure.lean mit 14 neuen Theoremen, zwei unveränderte Repository-Module, Audit.lean mit 17 ausgewählten Konstanten und drei absichtlich ungültige Negativkontrollen. Die drei zusätzlich auditierten bestehenden Hauptsätze heißen FIT001_checked, FIT002_checked und FIT003_checked. Diese Zahlen beschreiben den vorbereiteten Quellumfang, keine erfolgreichen Prüfergebnisse.

Upstream: Goldkelch/qik-vrt
 Commit: a86054139b49c13c5cd344753b248b46b5daf66f
 Lean: leanprover/lean4:v4.19.0
 Lake-Abhängigkeiten: nur die mitgelieferte Std-Bibliothek

PROVENANCE.json bindet die übernommenen Dateien mittels SHA-256. Der neue Zusatz ist über seine eigenen Dateien gebunden, nicht über den Upstream-Commit: Dieser Commit enthält die neuen Sätze nicht. Die Veröffentlichung älterer Belege ersetzt keinen neuen Kernellauf.

7.2 Vorgesehener ausführbarer Prüfweg

```
lean --version
lake --version
python3 verify.py
```

verify.py verlangt Version 4.19.0, führt lake clean und lake build aus und ruft danach lake env lean Audit.lean auf. Es verlangt für alle 17 Konstanten vollständige Berichte mit leerer Axiomenabhängigkeit. Die falschen Progress-, Stale-Transfer- und Effect-Dateien müssen mit einem sachbezogenen Lean-Fehler scheitern. Nur bei vollständigem Erfolg wird ein neues KERNEL_RECEIPT.json mit Quellen-, Binär-, Objekt- und Logbindungen erzeugt. [R3, R4]

Tatsächlich beobachtet:

lean --version: Exitcode 1; ‚failed to locate application‘.
 lake build mit angegebenem LEAN_SYSROOT: Exitcode 1; ‚could not detect the configuration of the Lake installation‘.

Der Fehler liegt vor der Prüfung des neuen Quelltexts. Es gibt deshalb weder einen erfolgreichen neuen Build noch einen bestätigten Axiom-Audit oder ausgeführte Negativkontrollen.

Die offizielle Distribution wurde lokal entpackt; ihr beobachteter SHA-256 steht in PROVENANCE.json. Da die Release-API keinen eigenen Digest lieferte, ist dieser Hash eine Bytebindung, kein unabhängiger Authentizitätsnachweis. Zugriffssperren wurden nicht umgangen.

Erst ein erfolgreicher Lauf in einer regulär ausführbaren Lean-4.19.0-Umgebung kann den offenen Kernelstatus ändern. Lake organisiert den Build; den Beleg liefern die geprüften Terme samt Voraussetzungen. [R3]

8 Aussagebilanz und offene Arbeit

ID	Hauptaussage	Nachweis / Status
EC-01	Passende finite Spuren sind verkettbar.	Induktionsbeweis im Text; Lean-Kandidat <code>trace_append</code> .
EC-02	C ist auf viablen Beständen extensiv, monoton und idempotent.	Herleitung in Abschnitt 3; neuer Kernelstatus offen.
EC-03	Ein frisches viables Element erlaubt echte Erweiterung abgeschlossener Bestände.	Bedingter Satz; Existenz des frischen Elements vorausgesetzt.
EC-04	Viable Nachfolger an jedem viablen Zustand tragen jeden endlichen Fortsetzungshorizont.	Bedingter Induktionsbeweis; kein realer Unendlichkeitsbeweis.
EC-05	Viabilität allein garantiert keine Fortsetzung.	Explizites Einzustands-Gegenmodell.
EC-06	Ein exakt gebundenes Receipt überträgt sich nicht automatisch auf ein anderes Subjekt.	Gleichheitsargument; keine neue Kryptografie.
EC-07	Transportbestätigung impliziert nicht generell den gewünschten Effekt.	Bool-Gegenmodell; physische Zuordnung separat.
EC-08	Ein konkretes QIK-VRT-SoC ist vollständig realisiert und geprüft.	Nicht hergeleitet, nicht implementiert, nicht behauptet.

8.1 Ergebnis

Die präzise mathematische These ist tragfähig: Unter expliziten Regeln kann ein Bestand abgeschlossen sein und nach Hinzunahme einer neuen zulässigen Grundlage einen echt größeren abgeschlossenen Bestand erzeugen. Abgeschlossenheit ist relativ; sie ist keine Behauptung von Allwissenheit. Die Aussage ist durch die angegebenen Beweise begründet, während ihre neue maschinelle Bestätigung in dieser Umgebung aussteht.

8.2 Was vor einer weitergehenden Aussage fehlt

Zuerst fehlen der erfolgreiche neue Kernellauf und die fachliche Prüfung, ob Definitionen und Voraussetzungen den beabsichtigten QIK-VRT-Anspruch angemessen abbilden. Für reale Systeme fehlen anschließend konkrete Verfeinerungs- und Messbelege. Physikalische Allgemeingültigkeit, unabhängige Reproduktion und wissenschaftlicher Konsens folgen weder aus dem Ausgangstext noch aus einer möglichen Kernelannahme.

Push, PR, Review, Approval, Merge und Publication sind jeweils eigene Vorgänge. PASS, FINAL_PASS und EFFECT_ACK_DONE sind getrennt zu evaluierende, scope-gebundene Zustände. Dieses lokale Artikelpaket setzt keinen dieser Zustände. Die Abgabe an den Auftraggeber ist keine Zenodo-Publikation und kein Peer Review.

9 Quellen und Beitragsprovenienz

[R1] Lohmann, I. (2026): `OperationalContinuation.lean`. QIK-VRT, unveränderter Quellauszug am angegebenen Commit. Definitionen von `ViableConnection` und `Survives` sowie FIT-001.

[Quelle öffnen](#)

[R2] Lohmann, I. (2026): `ConnectabilitySimulation.lean`. Derselbe Commit. `TypedTransitionSystem`, `ViableTrace` und die Simulationssätze FIT-002/3.

[Quelle öffnen](#)

[R3] Lean Language Reference, Version 4.19.0: Build Tools and Distribution / Lake. Offizielle Dokumentation des Buildwerkzeugs. Abgerufen am 15.09.2026.

[Quelle öffnen](#)

[R4] Theorem Proving in Lean 4: Axioms and Computation. Offizielle Darstellung von logischer Grundlage, zusätzlichen Axiomen und Rechenbedeutung. Abgerufen am 15.09.2026.

[Quelle öffnen](#)

[R5] Lamport, L. (1978): Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM* 21(7), 558-565. Autorensseitig bereitgestellte Fassung.

[Quelle öffnen](#)

[R6] Lean 4, offizielle Release-Distribution v4.19.0. Quelle des lokal bereitgestellten Linux-Laufzeitsystems; Herkunft und beobachteter Hash im Begleitpaket.

[Quelle öffnen](#)

Beitrags- und Rechteinweis

Ingolf Lohmann hat das QIK-VRT-Konzept, den Ausgangsartikel und die hier bearbeitete Fragestellung geliefert. OpenAI Codex hat die neue Präzisierung, den Lean-Zusatz, den Prüfharness und diese Artikelfassung erzeugt. Diese Trennung ist eine Herkunftsangabe und kein Nachweis historischer Priorität. Eine fachliche Annahme des neuen Kandidaten durch Ingolf Lohmann wird nicht vorweggenommen.

Die zwei wiederverwendeten Lean-Dateien behalten ihre Apache-2.0-Dateihinweise; der zugehörige Lizenztext ist im Paket enthalten. Durch diesen Artikel werden keine weiteren Rechte an anderen Repository-Bestandteilen eingeräumt. Das Begleitpaket enthält außerdem `CHANGE_NOTICE.md`, damit die Präzisierungen gegenüber den stärkeren Formulierungen der Ausgangsdiskussion sichtbar bleiben.

Zitiervorschlag

Lohmann, Ingolf (Konzept); OpenAI Codex (KI-assistierte Ausarbeitung): Abgeschlossen und anschlussfähig. Eine bedingte Formalisierung evidenzgebundener Zustandsarchitekturen mit einem Lean/Lake-Prüfpaket. Arbeitsartikel, Version 0.1, 15.09.2026. Unbegutachteter Kandidat; neuer Kernelstatus offen.